Software Architecture & System Specification Brief

Pozool Stays is an operations system of record for hybrid real estate managers handling both short-term vacation rentals and long-term residential leases. While currently serving under 1,000 operators with ~3,000 listings, the platform is preparing for expansion to 50,000+ active users and 250,000+ listings by Q4 2027. Student engineering teams must produce a Software Specification Document (SSD) and Architecture Design Document (ADD) addressing two vital business features.

1. BASELINE SYSTEM ARCHITECTURE & CONTEXT

Pozool Stays operates as a modular Ruby on Rails 8 monolith. The baseline system comprises:

For a complete layout of the system components and external services, refer to the **Annex A for Entity Relational Database and Infrastructure Diagram in Annex B.**

- **Application Core:** Ruby on Rails monolith running Action Pack, Active Record, Hotwire (Turbo/Stimulus), and Tailwind CSS for server-rendered operational views.
- **Transactional Data:** SQLite transactional database deployed on persistent volumes with SQLite Full-Text Search (FTS) tables and company-level Row-Level Security (RLS) policies.
- **Asynchronous Processing:** Solid Queue workers managing asynchronous jobs such as periodic external calendar syncs, tenant recurring lease billing, and status transitions.
- **Media & Assets:** S3-backed Active Storage integrated with an AWS Lambda event trigger that resizes variants into thumbnail buckets served via Amazon CloudFront.
- **Domain Entities:** Strict separation between Short-Term Rentals (Property to Listing to Reservation to CalendarBlock) and Long-Term Rentals (PropertyListing to Contract to Tenant to RentalPayment).

2. FEATURE 1: SCALABLE OTA CHANNEL INTEGRATION ENGINE

The Problem: Calendar syncing currently relies on polling outbound iCal feeds. At scale, iCal exhibits excessive synchronization latency (1-3 hours), lacks dynamic rate/pricing controls, cannot push listing restrictions or guest details, and routinely causes double-bookings. The system requires an enterprise-grade channel manager connecting Airbnb, Booking.com, VRBO, and Expedia.

Technical Specification Requirements

- **Integration Architecture & Trade-off Analysis:** Evaluate building first-party direct API integrations against integrating an established channel aggregator API (such as Channex or Beds24). Analyze API certification lifecycles, maintenance burdens, and PCI-DSS compliance boundaries.
- **Bi-Directional Event Synchronization:**
    - **Inbound (OTA to Pozool):** Webhook ingest endpoints with signature verification, idempotent consumer mechanics, deduplication keys, dead-letter queues (DLQ), and periodic reconciliation jobs.

    - **Outbound (Pozool to OTA):** Push synchronization architecture for Availability, Rates, Minimum Stay, and Restrictions (ARI data pipeline).
- **High-Concurrency Race Condition Prevention:** Detailed distributed locking, optimistic/pessimistic transactional reservation locks, and state transitions to prevent double-bookings when multiple channels book simultaneously.
- **Data & Scale Architecture:** Entity additions (ChannelAccount, ChannelListingMapping, ChannelSyncLog, ChannelWebhookEvent) and background queue scaling strategies capable of handling 250,000 listings without exhausting database connection pools or worker threads.

3. FEATURE 2: INTUITIVE, AI-POWERED DIRECT BOOKING SITE BUILDER

The Problem: Property managers incur 15%-20% commission overhead on major OTA channels. To build an independent direct-booking brand, operators need a zero-friction website builder that ingests their existing Pozool property data, photos, amenities, and house rules to publish high-converting, mobile-ready booking portals in minutes.

Technical Specification Requirements

- **Data Ingestion & Generative AI Engine:**
    - Pipeline to automatically aggregate existing structured records (Property, Listing, Active Storage assets, amenities, and policies).
    - LLM orchestration engine to generate search-optimized headline copy, persuasive neighborhood guides, and tailored guest FAQs with structured JSON validation and timeout fallback defaults.
- **Multi-Tenant Hosting & Domain Routing:** Architecture for serving tenant storefronts via custom subdomains or customer-owned custom domains with automated TLS certificate issuance (via Kamal proxy, Caddy, or CloudFront edge routing).
- **Direct Booking Checkout Pipeline:** End-to-end checkout flow interfacing with existing domain services (Reservations::Upsert and CalendarBlock) and PCI-compliant direct credit card tokenization (such as Stripe Elements) recording to Pozool's Payment ledger.

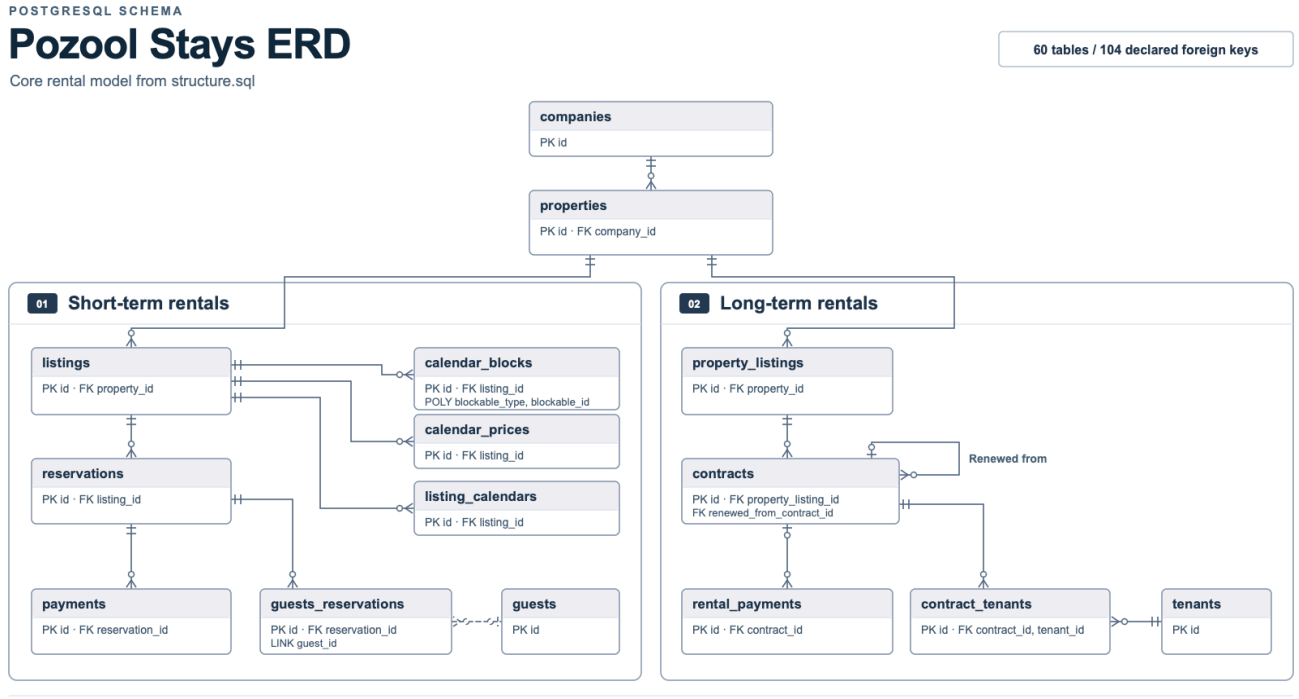
4. DELIVERABLES & TECHNICAL EVALUATION RUBRIC

Student engineering teams must author the Architecture Design Document (ADD) that will outline **ONE OF THE FEATURES** with their own approaches. Students are free to detail their approaches to implement this solution while also being expected to defend their criteria and choices. The project will be evaluated across three technical dimensions:

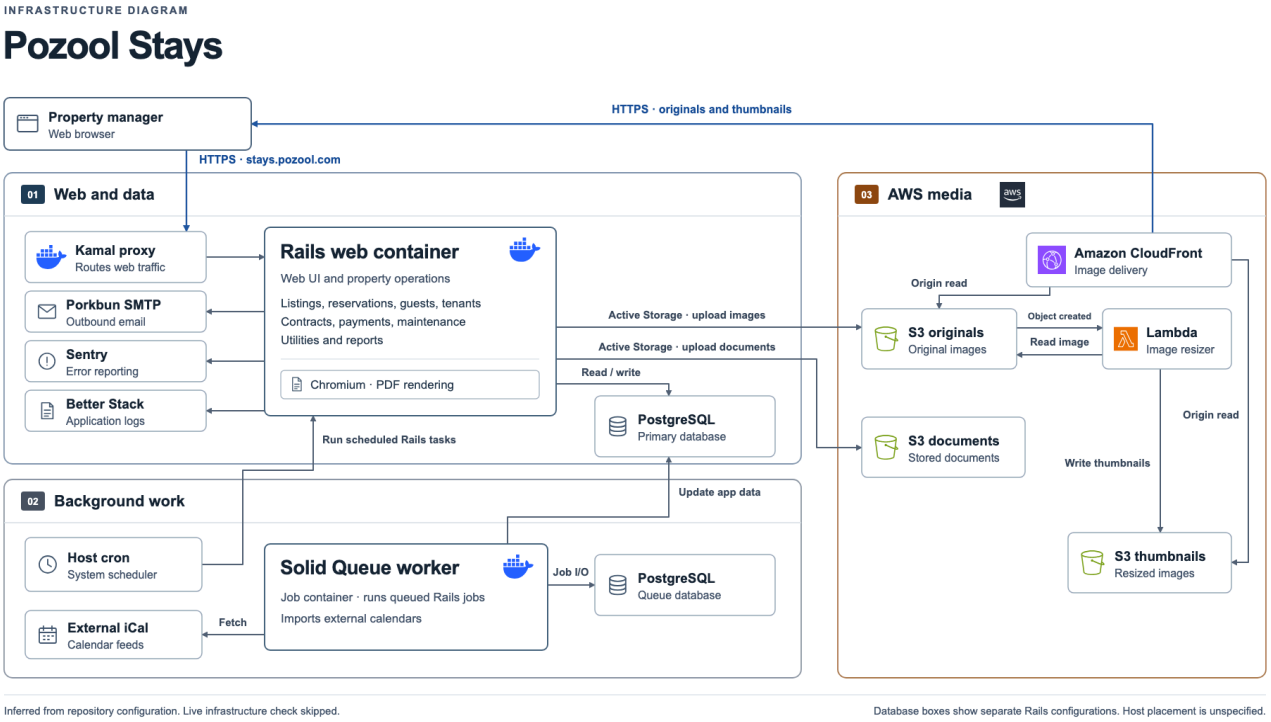
Assessment Category	Weight	Evaluation Criteria
System Modeling & C4 Diagrams	35%	Accurate extension of Pozool's existing diagrams and implement different diagrams we have already seem in class if necessary **Explicitly models new subsystems, background queues, webhook consumers, database**

Assessment Category	Weight	Evaluation Criteria
		interactions, edge routing, and external channel networks.
Technical Feasibility & Trade-offs	35%	Evidence-based architectural justification and trade-off analysis. Based on the feature of your choice: - Compares direct OTA APIs vs. aggregator middlewares - SSR vs. static edge site generation, and evaluates persistence/queue bottlenecks under 250k listings scale.
Final Presentation	30%	Defend their criteria and present to other students your results and analysis.

Annex A



Annex B


INFRASTRUCTURE DIAGRAM
Pozool Stays
Property manager
Web browser
HTTPS · originals and thumbnails
HTTPS · stays.pozool.com
01 Web and data
Kamal proxy
Routes web traffic
Porkbun SMTP
Outbound email
Sentry
Error reporting
Better Stack
Application logs
Rails web container
Web UI and property operations
Listings, reservations, guests, tenants
Contracts, payments, maintenance
Utilities and reports
Chromium · PDF rendering
Active Storage · upload images
Active Storage · upload documents
Read / write
PostgreSQL
Primary database
Run scheduled Rails tasks
03 AWS media
Amazon CloudFront
Image delivery
Origin read
S3 originals
Original images
Object created
Read image
Lambda
Image resizer
Origin read
S3 documents
Stored documents
Write thumbnails
S3 thumbnails
Resized images
02 Background work
Update app data
Host cron
System scheduler
Solid Queue worker
Job container · runs queued Rails jobs
Imports external calendars
Job I/O
PostgreSQL
Queue database
External iCal
Calendar feeds
Fetch
Inferred from repository configuration. Live infrastructure check skipped.
Database boxes show separate Rails configurations. Host placement is unspecified.